ASSIGNMENT 1

Design, Implement and Validate a Payment Calculator for a Village Milk Co-operative

Course Outcome CO1 – Python Programming

SUBMITTED BY:- M SANTHOSH KUMAR(192412268)

Scenario and Problem Statement

Ksheera Village Milk Producers' Co-operative receives milk from farmers twice a day. For each entry, the clerk records farmer ID, litres, fat percentage, SNF percentage and shift. The base price is Rs 30.00 per litre at 3.5% fat and 8.5% SNF. Each 0.1% fat step changes the rate by Rs 0.40, and each 0.1% SNF step changes it by Rs 0.30. Payments are settled every ten days.

The design must avoid floating-point money errors, reject invalid entries, centralise rates, support testing and remain maintainable when the number of farmers increases.

Design decisions used throughout

- Use Decimal constructed from strings for monetary calculations; never create Decimal from a binary float.
- Round the calculated rate and each farmer payment to two decimal places using ROUND_HALF_UP as the explicit currency rule.
- Treat fat and SNF step counts as nearest whole 0.1% steps, as required by the original brief.
- Validate litres > 0, fat between 2.0% and 7.0%, and SNF between 0.0% and 15.0% (SNF limits are an implementation assumption because the brief does not specify them).
- Keep rate values in one rate card/module. Store entries in a list when audit trails and reprints are required; use a running total when only the total is needed.

TASK 1 – Understand the Problem and Identify the Data

Value	Type	Purpose and justification
farmer_id	str	Identifier; text preserves leading zeroes and may contain letters.
litres	Decimal	Milk quantity may be fractional; Decimal supports consistent decimal arithmetic.
fat_pct	Decimal	Fractional percentage measured by analyser.
snf_pct	Decimal	Fractional solids-not-fat percentage.
shift	str / enum-like value	Identifies morning or evening collection and supports audit queries.
rate_per_litre	Decimal	Computed rate after fat and SNF adjustments.
amount	Decimal	Litres multiplied by rate, rounded to currency precision.
ten_day_total	Decimal	Sum of rounded individual payments for the ten-day settlement.
BASE_RATE, FAT_STEP_VALUE, SNF_STEP_VALUE	Decimal constants	Rates are stable during one run and should be defined once.
FAT_REFERENCE, SNF_REFERENCE, STEP_SIZE	Decimal constants	Reference percentages and 0.1% step size used by the formula.

Rules: farmer ID and shift must not be blank; litres must be greater than zero; fat must be within 2.0–7.0%; numeric fields must parse successfully; and each payment must be rounded to paise before being included in the settlement total. The 2.0–7.0% fat range is given as an example in the brief. The SNF range above is an explicit assumption and should be confirmed with the co-operative.

Assumptions: one record represents one farmer's collection for one shift; rate changes are applied uniformly during a run; no deductions, penalties or bonuses apply in the original scenario; totals are the sum of individually rounded payments.

Alternative concept and complexity

Using separate variables such as litres1, fat1, snf1 through litres300, fat300, snf300 creates many names and requires editing code when the number of farmers changes. A list of records such as [(litres, fat, snf), ...] supports loops and arbitrary n. Processing n records takes O(n) time. A list of n records uses O(n) space; a running total uses O(1) extra space if individual records do not need to be retained.

TASK 2 – Explore Values and Types in the Interpreter

```
>>> type(10)
<class 'int'>
>>> type(10.0)
<class 'float'>
>>> type("10")
<class 'str'>
>>> type(True)
<class 'bool'>
>>> type([10, 8.5])
<class 'list'>
>>> 7 / 2
3.5
>>> 7 // 2
3
>>> 7 % 2
1
>>> -7 // 2
-4
>>> 0.1 + 0.2 == 0.3
False
```

The / operator performs true division, // floors the quotient, and % returns the remainder. Because floor moves toward negative infinity, -7 // 2 is -4 rather than -3. Binary floating-point cannot represent many decimal fractions exactly, so 0.1 + 0.2 is slightly different from 0.3.

Complete the Code – Decimal total

```
from decimal import Decimal

payments = ["33.40"] * 1000 + ["0.10"] * 3

def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total

def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total

print("float  :", total_float(payments))
```

```
print("Decimal:", total_decimal(payments))
print(0.1 + 0.2 == 0.3,
      Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
```

Expected result: float accumulation is approximately 33400.30000000061; Decimal accumulation is exactly 33400.30; the comparisons print False and True. Decimal(float(amount)) first imports the binary approximation already stored in the float, so the Decimal value may contain many unwanted digits. Decimal(amount) where amount is a string preserves the intended decimal value.

Alternative analysis: integer paise is exact when all amounts are represented to two decimal places (Rs 33.40 becomes 3340 paise). Decimal is also exact for decimal inputs. float plus round() after each addition is not a reliable general accounting method because rounding the accumulated approximation does not remove every possible error. Each approach takes O(n) time for n payments. Integer and float values are compact; Decimal values usually use more memory. Use sys.getsizeof(value) to measure on the Python version used for submission.

```
import sys
from decimal import Decimal
print(sys.getsizeof(3340))
print(sys.getsizeof(33.40))
print(sys.getsizeof(Decimal("33.40")) )
```

TASK 3 – Choose Variables and Name the Constants

```
from decimal import Decimal

BASE_RATE = Decimal("30.00")
FAT_REFERENCE = Decimal("3.5")
SNF_REFERENCE = Decimal("8.5")
STEP_SIZE = Decimal("0.1")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
MIN_FAT = Decimal("2.0")
MAX_FAT = Decimal("7.0")
```

Python naming convention uses lowercase_with_underscores for variables and functions and UPPER_CASE for constants. Python does not enforce true constants; uppercase is a convention that signals a value should not be reassigned. Python is dynamically typed: a variable can refer to an integer and later to a string, for example x = 5 followed by x = 'five'. Multiple assignment can swap values without a temporary variable: a, b = b, a.

If rates are magic numbers embedded in k expressions, a revision may require changing k places and risks inconsistent updates. A named constant or RATE_CARD dictionary centralises values. Lookup adds negligible constant-time work and small memory overhead; runtime complexity remains O(1) per calculation.

TASK 4 – Write Correct Expressions

Let fat_steps = round((fat - FAT_REFERENCE) / STEP_SIZE) and snf_steps = round((snf - SNF_REFERENCE) / STEP_SIZE). Then rate = BASE_RATE + fat_steps × FAT_STEP_VALUE + snf_steps × SNF_STEP_VALUE. Payment = litres × rate, rounded to two decimal places.

```
fat_steps = round((fat - FAT_REFERENCE) / STEP_SIZE)
snf_steps = round((snf - SNF_REFERENCE) / STEP_SIZE)
rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
amount = (litres * rate).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
```

Floating-point example: (4.2 - 3.5) / 0.1 may evaluate to 7.000000000000001 because decimal fractions are approximated in binary. round() produces the intended integer step count in this case. Operator precedence

places exponentiation before unary minus in expressions such as -2**2, which is -(2**2) = -4; (-2)**2 = 4. Comparison chaining such as 2.0 <= fat <= 7.0 is equivalent to two comparisons joined by and.

For negative values, int() truncates toward zero, while math.floor() rounds downward. round() rounds to the nearest integer and Python uses ties-to-even for exact halfway cases. For fat 4.2, the mathematical step is +7: round, int and floor all give 7 for the displayed value. For fat 3.1, the mathematical step is -4: round gives -4, int may give -3 if the computed value is -3.999999999999999, and floor gives -4. This illustrates why a decimal-based step calculation is preferable. Each method is O(1) time and O(1) space.

Expression	Possible result	Explanation
0.1 + 0.2 == 0.3	False	Binary floating-point representation.
(4.2 - 3.5) / 0.1	7.000000000000001	Intermediate binary approximation.
-7 // 2	-4	Floor division rounds down.
-22**	-4	Exponentiation binds before unary minus.

TASK 5 – Read Input and Convert Types

```
def read_decimal(prompt):
    while True:
        text = input(prompt).strip()
        try:
            return Decimal(text)
        except Exception:
            print("Invalid number. Enter a value such as 8.5.")

# Examples:
# "8,5" -> rejected by Decimal in the standard decimal-point format
# "eight" -> rejected
# " 8.5 " -> accepted after strip()
```

input() always returns str. Convert identifiers and shift labels as text, and convert numeric entries to Decimal after stripping whitespace. A try/except around Decimal(text) is clearer and more robust than trying to validate every numeric form with string methods. A comma decimal such as '8,5' is rejected unless the program explicitly supports locale-specific formats. Formatting can use f-strings, for example f'{amount:8.2f}', to align currency values.

Both string checks and try/except require O(1) time per entry for short numeric strings. try/except directly handles whitespace and malformed text, and reports invalid input without crashing.

TASK 6 – Organise the Logic into Functions

```
from decimal import Decimal, ROUND_HALF_UP

CENT = Decimal("0.01")

def rate_per_litre(fat, snf):
    """Return the price per litre for the supplied fat and SNF values."""
    fat_steps = ((fat - FAT_REFERENCE) / STEP_SIZE).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    snf_steps = ((snf - SNF_REFERENCE) / STEP_SIZE).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
    return rate.quantize(CENT, rounding=ROUND_HALF_UP)
```

```
def payout(litres, fat, snf):
    """Return one farmer's payment rounded to paise."""
    return (litres * rate_per_litre(fat, snf)).quantize(
        CENT, rounding=ROUND_HALF_UP)

def ten_day_total(payments):
    """Return the sum of individual payments."""
    return sum(payments, Decimal("0.00")).quantize(CENT, rounding=ROUND_HALF_UP)
```

A function should return a value so another function or test can reuse it; printing is only a presentation action. Local variables exist inside the function call. Constants are read by functions but should not be reassigned there. Docstrings describe purpose, parameters and return values. A single long script duplicates logic and is harder to test; lambda expressions are unsuitable for multi-step validation and documentation. Function-call overhead is small compared with processing large input files; total work remains O(n).

TASK 7 – Build a Module

```
# milkrate.py
"""Rate card and calculation functions for the milk co-operative."""
from decimal import Decimal, ROUND_HALF_UP

BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
FAT_REFERENCE = Decimal("3.5")
SNF_REFERENCE = Decimal("8.5")
STEP_SIZE = Decimal("0.1")
CENT = Decimal("0.01")

def rate_per_litre(fat, snf):
    """Return rate per litre rounded to two decimal places."""
    fat_steps = ((fat - FAT_REFERENCE) / STEP_SIZE).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    snf_steps = ((snf - SNF_REFERENCE) / STEP_SIZE).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
    return rate.quantize(CENT, rounding=ROUND_HALF_UP)

def payout(litres, fat, snf):
    """Return payment rounded to two decimal places."""
    return (litres * rate_per_litre(fat, snf)).quantize(
        CENT, rounding=ROUND_HALF_UP)

if __name__ == "__main__":
    for litres, fat, snf in [
        (Decimal("10"), Decimal("3.5"), Decimal("8.5")),
        (Decimal("8.5"), Decimal("4.2"), Decimal("8.7")),
        (Decimal("12"), Decimal("3.1"), Decimal("8.3"))
    ]:
        rate = rate_per_litre(fat, snf)
        pay = payout(litres, fat, snf)
        print(f"{litres:>5} L fat {fat} snf {snf} rate {rate:>6} pay {pay:>8}")
```

```
# test_milkrate.py
from decimal import Decimal
import milkrate as mr

print(mr.rate_per_litre(Decimal("4.2"), Decimal("8.7")))
print(mr.payout(Decimal("8.5"), Decimal("4.2"), Decimal("8.7")))
```

import milkrate exposes names through the module; from milkrate import payout imports that function directly; import milkrate as mr uses an alias. The if __name__ == '__main__': block runs only when the file is executed directly. When imported, __name__ is 'milkrate', so the demonstration table is not printed. Without the guard, demonstration code would run during import. Python searches locations listed in sys.path, including the script/current directory. Modules are cached in sys.modules after the first import.

Copying functions into every script means every copy must be updated after a rate change. Importing a module has a one-time loading cost and then makes shared functions reusable.

TASK 8 – Plan the Algorithm Before Coding

Pseudocode

```
BEGIN
  SET ten_day_total = 0.00
  FOR each entry received
    READ farmer_id, litres, fat, snf, shift
    IF input is not numeric OR farmer_id/shift is blank
       OR litres <= 0 OR fat < 2.0 OR fat > 7.0
       OR snf < 0.0 OR snf > 15.0 THEN
         REPORT invalid entry
    ELSE
         fat_steps = nearest_integer((fat - 3.5) / 0.1)
         snf_steps = nearest_integer((snf - 8.5) / 0.1)
         rate = 30.00 + fat_steps*0.40 + snf_steps*0.30
         rate = round rate to currency precision
         payment = round(litres * rate to paise)
         ten_day_total = ten_day_total + payment
         STORE entry and payment for audit
    END IF
  END FOR
  DISPLAY valid payments and ten_day_total
END
```

Flowchart (text form)

```
[Start] -> [Read entry] -> <All fields valid?>
          No -> [Show error / reject entry] -> [Read next entry]
          Yes -> [Calculate steps and rate] -> [Calculate rounded payment]
               -> [Add payment to ten-day total] -> [Store record]
               -> [Read next entry]
After last entry -> [Display total] -> [End]
```

Hand trace for three entries

Entry	Litres	Fat	SNF	Rate calculation	Payment
1	10	3.5	8.5	30.00	300.00
2	8.5	4.2	8.7	30 + 7(0.40) + 2(0.30) = 33.40	283.90
3	12	3.1	8.3	30 - 4(0.40) - 2(0.30) = 27.80	333.60
Total					917.50

Both storing records and maintaining a running total take O(n) time. Storing all entries uses O(n) space and supports audit, corrections and reprinting. A running total uses O(1) extra space and is sufficient when only a total is required; in a real co-operative, records should normally be retained for audit.

TASK 9 – Implement and Test the Calculator

```
# milk_calculator.py
from decimal import Decimal, ROUND_HALF_UP

BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
FAT_REFERENCE = Decimal("3.5")
SNF_REFERENCE = Decimal("8.5")
STEP_SIZE = Decimal("0.1")
MIN_FAT, MAX_FAT = Decimal("2.0"), Decimal("7.0")
MIN_SNF, MAX_SNF = Decimal("0.0"), Decimal("15.0")
CENT = Decimal("0.01")

def rate_per_litre(fat, snf):
    """Calculate the rate per litre using nearest 0.1% steps."""
    fs = ((fat - FAT_REFERENCE) / STEP_SIZE).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    ss = ((snf - SNF_REFERENCE) / STEP_SIZE).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    return (BASE_RATE + fs*FAT_STEP_VALUE + ss*SNF_STEP_VALUE).quantize(
        CENT, rounding=ROUND_HALF_UP)

def payout(litres, fat, snf):
    """Validate and calculate one payment."""
    if litres <= 0:
        raise ValueError("Litres must be positive")
    if not MIN_FAT <= fat <= MAX_FAT:
        raise ValueError("Fat must be between 2.0 and 7.0")
    if not MIN_SNF <= snf <= MAX_SNF:
        raise ValueError("SNF must be between 0.0 and 15.0")
    return (litres * rate_per_litre(fat, snf)).quantize(
        CENT, rounding=ROUND_HALF_UP)

def ten_day_total(entries):
    """Return total of (litres, fat, snf) records."""
    total = Decimal("0.00")
    for litres, fat, snf in entries:
        total += payout(litres, fat, snf)
    return total.quantize(CENT, rounding=ROUND_HALF_UP)

def run_tests():
    """Run repeatable assertion-based tests."""
    assert rate_per_litre(Decimal("3.5"), Decimal("8.5")) == Decimal("30.00")
    assert payout(Decimal("10"), Decimal("3.5"), Decimal("8.5")) == Decimal("300.00")
    assert rate_per_litre(Decimal("4.2"), Decimal("8.7")) == Decimal("33.40")
    assert payout(Decimal("8.5"), Decimal("4.2"), Decimal("8.7")) == Decimal("283.90")
    assert rate_per_litre(Decimal("3.1"), Decimal("8.3")) == Decimal("27.80")
    assert payout(Decimal("12"), Decimal("3.1"), Decimal("8.3")) == Decimal("333.60")
    assert rate_per_litre(Decimal("2.0"), Decimal("8.5")) == Decimal("24.00")
    assert rate_per_litre(Decimal("7.0"), Decimal("8.5")) == Decimal("44.00")
    try:
        payout(Decimal("-1"), Decimal("3.5"), Decimal("8.5"))
        assert False, "Negative litres should fail"
    except ValueError:
        pass
    try:
        payout(Decimal("1"), Decimal("7.1"), Decimal("8.5"))
        assert False, "Out-of-range fat should fail"
```

```
        except ValueError:
            pass
        print("All tests passed")

    if __name__ == "__main__":
        run_tests()
```

Test	Input	Expected result	Validation
Standard	10 L, fat 3.5, SNF 8.5	Rate 30.00; pay 300.00	Pass by formula
Both above	8.5 L, fat 4.2, SNF 8.7	Rate 33.40; pay 283.90	Pass by formula
Both below	12 L, fat 3.1, SNF 8.3	Rate 27.80; pay 333.60	Pass by formula
Minimum fat	fat 2.0, SNF 8.5	Rate 24.00	Pass by formula
Maximum fat	fat 7.0, SNF 8.5	Rate 44.00	Pass by formula
Negative litres	-1 L	ValueError	Pass if exception raised
Invalid fat	7.1%	ValueError	Pass if exception raised
Hand-check set	The first three entries above	Total Rs 917.50	Pass by hand calculation

Assertion-based tests can be rerun automatically after every change and are repeatable. Manual testing is useful for exploratory checks but takes longer and is easier to perform inconsistently. The code above is a reference implementation; execute it locally and capture the actual terminal output for submission.

TASK 10 – Compare Expectations with Execution

The following script compares float and Decimal accumulation and measures time. Run it on the submission computer and copy the actual values into the table; timing varies by hardware, operating system and Python version.

```
from decimal import Decimal
from time import perf_counter

def benchmark(n):
    amounts = ["33.40"] * n
    start = perf_counter()
    ftotal = 0.0
    for x in amounts:
        ftotal += float(x)
    float_seconds = perf_counter() - start

    start = perf_counter()
    dtotal = sum((Decimal(x) for x in amounts), Decimal("0.00"))
    decimal_seconds = perf_counter() - start
    print(n, ftotal, dtotal, float_seconds, decimal_seconds)

for n in (300, 6000, 150000):
    benchmark(n)
```

Payments	Expected Decimal total if each is Rs 33.40	Float total / difference	Measured time
300	Rs 10,020.00	Record actual output	Fill after execution
6,000	Rs 200,400.00	Record actual output	Fill after execution
150,000	Rs 5,010,000.00	Record actual output	Fill after execution

The Decimal total should match the exact decimal amount. Float totals may differ by a small binary rounding error. For bank settlement, round each farmer's payment to paise and then sum those rounded payments; rounding only the final total can differ when individual unrounded payments contain fractions of a paisa. The

rate-table optimisation can cache each distinct (fat, snf) pair: lookup is average O(1), compared with a few arithmetic operations for recalculation, and extra space is O(k) for k unique pairs. Measure with time.perf_counter() or timeit rather than assuming a speed-up.

TASK 11 – Recommend and Defend the Final Design

Recommendation: retain the modular design, but use Decimal inputs and explicit currency rounding; centralise the rate card; validate all entries; preserve a record for each farmer and shift; and use automated tests. A rate revision is made in one rate-card location, then the test suite is run before deployment. The design is O(n) for n records and is suitable for 150,000 entries in principle, subject to file/database handling, audit requirements and operational testing.

Limitations: the brief does not provide an official SNF validation range, seasonal effective-date policy, duplicate-entry rules or tax/deduction rules. These must be confirmed with the co-operative. Decimal is slower than float but is appropriate for exact decimal accounting. A JSON/text rate file would separate configuration from code and avoid code edits for every rate revision, but it must be validated at start-up; reading t values takes O(t) time and O(t) space. Editing a configuration file is easier, but versioning, access control and effective dates must be managed.

TASK 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Design

A. Impact analysis

The analyser now reports fat and SNF to two decimal places. From April through June, the base rate becomes Rs 32.00 and each fat step is Rs 0.45; the SNF step remains Rs 0.30 because no changed SNF rate is specified. A farmer supplying more than 500 litres in ten days receives a bonus of Rs 0.50 per litre. Constants and rate selection must change, step calculations must use the analyser precision while preserving the defined 0.1% pricing step, and ten-day totals must be available before applying the volume bonus.

B. Evaluate original design

The original functions and module remain useful, but the fixed rate constants should be replaced by a rate-card parameter selected by season. Two-decimal readings should be converted from strings to Decimal to avoid binary representation issues. Because the price changes every 0.1%, the calculation remains based on the number of 0.1% steps; two-decimal measurements can produce fractional step counts, which the stated nearest-whole-step rule rounds to the nearest integer.

C. Revised implementation

```
from decimal import Decimal, ROUND_HALF_UP

CENT = Decimal("0.01")
STEP = Decimal("0.1")
FAT_REF = Decimal("3.5")
SNF_REF = Decimal("8.5")

RATE_CARDS = {
    "standard": {
        "base": Decimal("30.00"),
        "fat_step": Decimal("0.40"),
        "snf_step": Decimal("0.30")
    },
    "april_june": {
        "base": Decimal("32.00"),
        "fat_step": Decimal("0.45"),
```

```
        "snf_step": Decimal("0.30")
    }
}

def nearest_step(value):
    """Return nearest whole 0.1 percentage-point step."""
    return ((value / STEP).quantize(Decimal("1"), rounding=ROUND_HALF_UP))

def seasonal_rate(fat, snf, season="standard"):
    """Return rate for the selected season."""
    card = RATE_CARDS[season]
    fat_steps = nearest_step((fat - FAT_REF) / STEP)
    snf_steps = nearest_step((snf - SNF_REF) / STEP)
    rate = card["base"] + fat_steps*card["fat_step"] + snf_steps*card["snf_step"]
    return rate.quantize(CENT, rounding=ROUND_HALF_UP)

def farmer_payment(litres, fat, snf, season="standard"):
    """Return payment before any ten-day volume bonus."""
    return (litres * seasonal_rate(fat, snf, season)).quantize(
        CENT, rounding=ROUND_HALF_UP)

def ten_day_settlement(entries, season="standard"):
    """Return base total, bonus and final total for entries (litres, fat, snf)."""
    base_total = sum(
        (farmer_payment(l, f, s, season) for l, f, s in entries),
        Decimal("0.00")
    ).quantize(CENT, rounding=ROUND_HALF_UP)
    total_litres = sum((l for l, _, _ in entries), Decimal("0"))
    bonus = (total_litres * Decimal("0.50")).quantize(CENT, rounding=ROUND_HALF_UP)         if
total_litres > Decimal("500") else Decimal("0.00")
    return base_total, bonus, (base_total + bonus).quantize(CENT)

# Demonstration
sample = [(Decimal("501.00"), Decimal("3.56"), Decimal("8.52"))]
print("Standard:", seasonal_rate(Decimal("3.56"), Decimal("8.52"), "standard"))
print("April-June:", seasonal_rate(Decimal("3.56"), Decimal("8.52"), "april_june"))
print("Settlement:", ten_day_settlement(sample, "april_june"))
```

Interpretation of bonus: this implementation applies the Rs 0.50-per-litre bonus when the farmer's combined ten-day volume exceeds 500 litres. It assumes the bonus is calculated on all litres supplied in that period, not only litres above 500. If the co-operative intends a different rule, the bonus function must be adjusted.

D. Evidence-based defence and test cases

Test case	Calculation	Expected result
Standard season reference milk	Base 30.00; fat 3.50; SNF 8.50	Rate Rs 30.00/L
April–June reference milk	Base 32.00; fat 3.50; SNF 8.50	Rate Rs 32.00/L
April–June fat 3.56, SNF 8.52	Fat offset 0.06% rounds to 1 step; SNF offset 0.02% rounds to 0 steps	Rate Rs 32.45/L
Ten-day total 500.00 L	Not more than 500 L	No bonus
Ten-day total 501.00 L	501 × Rs 0.50	Bonus Rs 250.50

Defence: MODIFY is preferable to a full redesign. The existing functions, validation and module structure remain suitable; the rate card is selected through a season parameter, and a separate settlement function applies the volume bonus. This limits duplicated code and keeps seasonal updates testable. Before production, the co-operative must confirm the intended rounding rule for two-decimal analyser readings, whether the SNF rate changes seasonally, and the exact bonus eligibility calculation.

Overall Conclusion

The recommended calculator uses explicit data types, Decimal arithmetic, named constants, reusable functions, a separate module, input validation, repeatable assertions and documented assumptions. Its processing time is O(n), and it can handle the centre, block and district-union data sizes in a batch workflow. Actual execution screenshots and measured timing results must be added by the student before submission.

Submission checklist

- Fill in student name and register number.
- Create the required GitHub repository and add its URL to this report.
- Save each task's code in a separate Task01–Task12 folder.
- Execute all code and replace illustrative/expected outputs with screenshots from your own run.
- Run timing experiments for 300, 6,000 and 150,000 payments and enter measured results.
- Add the required UML diagrams and any additional course-specific pseudocode format required by SIMATS Engineering.
- Commit progressively, at least one commit per task.